

Support de présentation de la méthodologie

Présentation de la **logique, des étapes méthodologiques, et des techniques utilisées** pour accomplir le projet.

OBJECTIFS PEDAGOGIQUES

- Charger des données dans une base de données
- Créer un schéma d'une base de données
- Rédiger des scripts simples en SQL pour aider à la prise de décision business
- Découvrir la **création** et la **gestion de bases de données**.

[illegible]

DECOUVERTE DES DONNEES

- 1 Exploration et complétude des fichiers CSV contenant les **contrats clients** et les **données géographiques des régions**.
- 2 Analyse des types de données présentes (champs, formats). Dans le dictionnaire des données, description des données pour chaque ligne. Prendre le temps de comprendre chaque donnée.
- 3 Possibilité d'utilisation d'Excel pour examiner les données via des filtres et des tableaux croisés dynamiques.
- 4 Bien comprendre les variables et leur rôle avant de passer à la modélisation.

COMPLETUDU DU DICTIONNAIRE DE DONNEES

Etapes

- Correspondance des colonnes entre les fichiers CSV et le dictionnaire des données.
- Définition des types de variables : INTEGER, VARCHAR, FLOAT, etc.
- Ajout des contraintes (longueur maximale) et description des données pour chaque colonne.

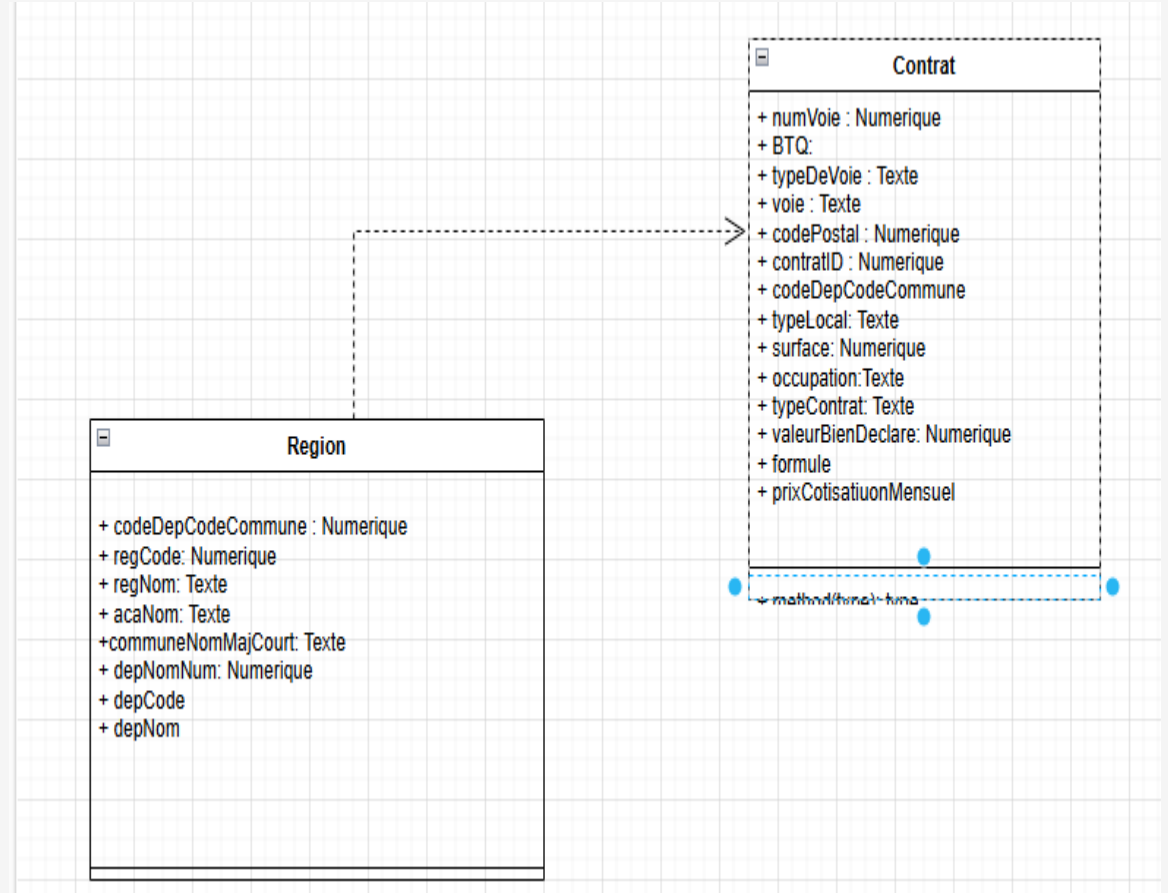
Outils utilisés

- Excel, Openclassrooms et Codecademy, pour documenter les colonnes et faciliter la compréhension.

	Nom des colonnes	Type de données	Taille	Clé	Description
CONTRAT.CSV	Contrat_ID	INT		Clé primaire	Id unique pour les contrats
	No_voie	INT			Numéro dans la voie pour l'adresse du logement assuré
	B_T_Q	CHAR	1		Indicateur éventuel de répétition pour l'adresse du logement assuré sur un caractère
	Type_de_voie	VARCHAR			Type de voie pour l'adresse du logement assuré: rue, av (Avenue), rte (Route), ...
	Voie	VARCHAR			Libellé de la voie pour l'adresse du logement assuré
	Code_dep_code_commune	VARCHAR		Clé secondaire	Concaténation du code département et code commune pour avoir une clé unique
	Code_postal	VARCHAR	5		Code postal pour l'adresse du logement assuré
	Surface	INT			Taille
	Type	VARCHAR	20		Type de logement: Appartement ; Maison ..
	Occupation	VARCHAR	13		Statut de la personne occupante: Propriétaire, Locataire
	Type contrat	VARCHAR	30		Type contrat: Residence principale; Mise en location
	formule	VARCHAR	10		Type d'assurance : Intégral, Classique
REGION.CSV	Valeur déclaré bien	INT	(0-50000)		La valeurs des biens compris entre 0-50000
	Prix cotisation mensuel	INT			Prix mensuel de la cotisation d'assurance
	Code_dep_code_commune	VARCHAR		Clé primaire	Concaténation du code département et code commune pour avoir une clé unique
	reg_code	INT			Code de la région
	reg_nom	VARCHAR	30		Nom de la région
	aca_nom	VARCHAR	30		Le nom de l'Académie
	dep_nom	VARCHAR	30		Le nom du département
	com_nom_maj_court	VARCHAR	30		Le nom de la Commune en majuscule et abrégé si il est trop long
	dep_code	INT			Code du département
	dep_nom_num	VARCHAR			Le nom du département + le numéro

MODELISATION DE LA BDD EN TROIS ETAPES

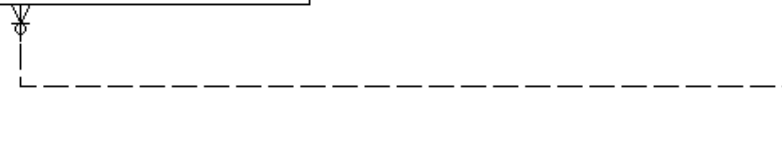
- 1 **Modèle conceptuel de donnée MCD.** Chercher les concepts présents et les associations qui existe entre les données. Pour dessiner le modèle conceptuel sous forme de diagramme de classes UML. on a utilisé ici diagramme sur drawio.com (ci-après)
- 2 **Le Modèle logique de données MLD modèle relationnel (formaté en tableau)** Analyse des relations entre les données pour modéliser les tables.
- 3 **Modèle physique de donnée MPD.** On implémente MLD> SGBDR avec le langage SQL



2 Le modèle relationnel

- 1 Création du schéma relationnel normalisé via SQL POWER ARCHITECT
- 2 Création des tables Contrat, Région, Identification des [primary key]
- 3 Identification de la clé commun [Foreign key] Code_dep_code_commune

Contrat	
Contrat_ID: INTEGER	NOT NULL [PK]
No_voie: INTEGER	NOT NULL
B_T_Q: VARCHAR	NOT NULL
Type_de_voie: VARCHAR	NOT NULL
Voie: VARCHAR	NOT NULL
Code_Postal: INTEGER	NOT NULL
Code_dep_code_commune: VARCHAR	NOT NULL [FK]
Type: VARCHAR(20)	NOT NULL
Surface: INTEGER	NOT NULL
Occupation: VARCHAR(13)	NOT NULL
Type_de_contrat: VARCHAR(10)	NOT NULL
Valeur_bien_declare: VARCHAR	NOT NULL
Formule: VARCHAR(10)	NOT NULL
Prix_cotisation_mensuel: INTEGER	NOT NULL



Region	
Code_dep_code_commune: VARCHAR	NOT NULL [PK]
Reg_code: INTEGER	NOT NULL
Reg_nom: VARCHAR(30)	NOT NULL
Aca_nom: VARCHAR(30)	NOT NULL
Dep_nom: VARCHAR	NOT NULL
Commune_nom_maj_court: VARCHAR(30)	NOT NULL
Dep_code: INTEGER	NOT NULL
Dep_nom_num: VARCHAR(30)	NOT NULL

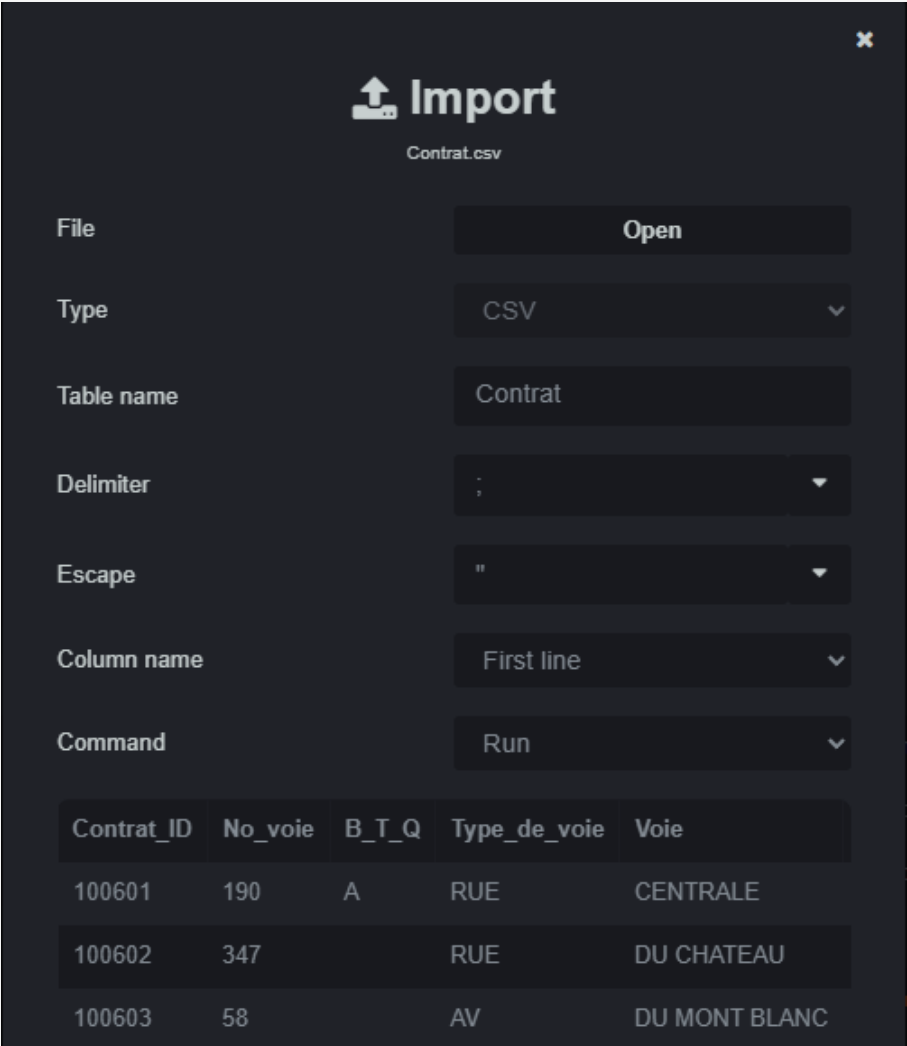
3 Le modèle physique des données (SGBDR)

Création de la base de données, traduction du modèle relationnel en code SQL par le SGBDR de notre choix (Postgre SQL, My SQL, Oracle, SQLite , etc...)

1 choix de travailler sur le SGBDR **SQLite Online** pour son accessibilité et sa simplicité d'utilisation dans ce contexte

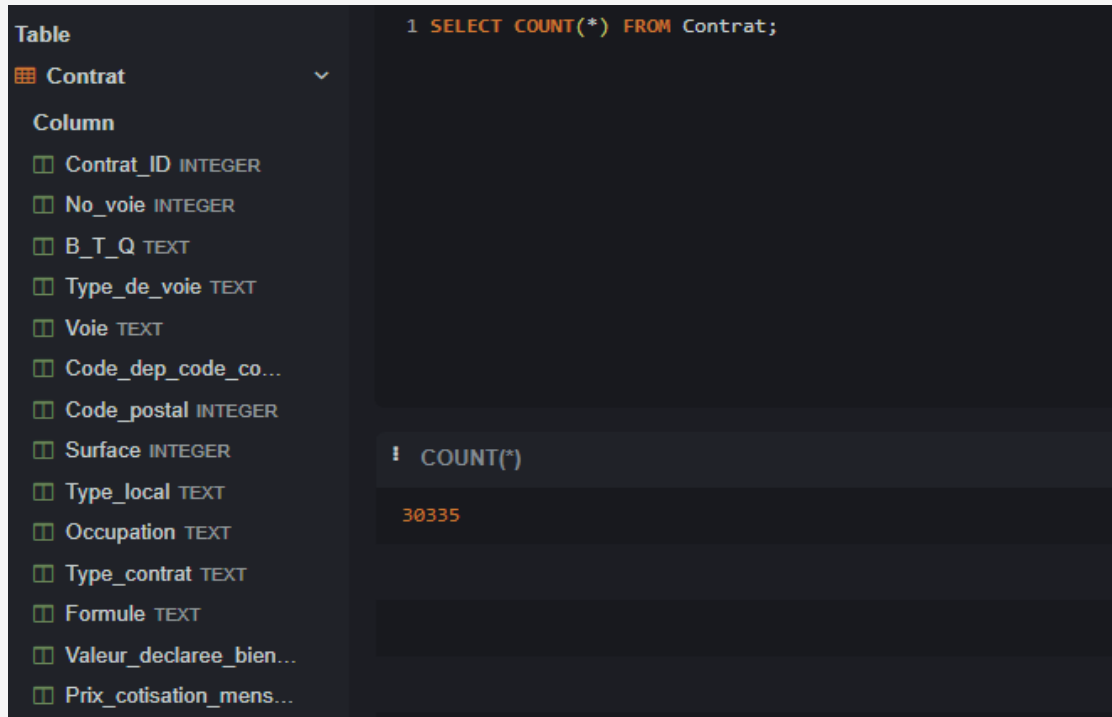
2 Import l'un après l'autre des 2 fichiers CSV dans la base de données SQLite Online

3 Sélection du *Delimiter* et Sélection du *Column name* pour nommer les colonnes.



Contrat_ID	No_voie	B_T_Q	Type_de_voie	Voie
100601	190	A	RUE	CENTRALE
100602	347		RUE	DU CHATEAU
100603	58		AV	DU MONT BLANC

Vérification chargement des données



The screenshot shows a database interface with a sidebar on the left listing the columns of the 'Contrat' table. The main area displays a SQL query and its result.

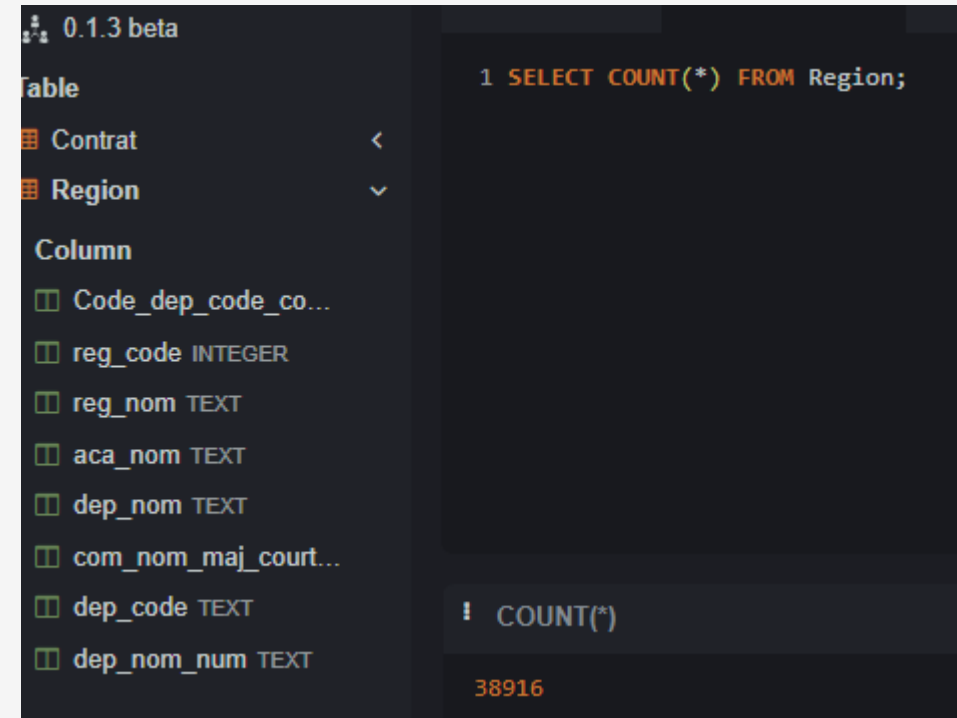
Table
Contrat

Column
Contrat_ID INTEGER
No_voie INTEGER
B_T_Q TEXT
Type_de_voie TEXT
Voie TEXT
Code_dep_code_co...
Code_postal INTEGER
Surface INTEGER
Type_local TEXT
Occupation TEXT
Type_contrat TEXT
Formule TEXT
Valeur_declaree_bien...
Prix_cotisation_mens...

```
1 SELECT COUNT(*) FROM Contrat;
```

! COUNT(*)
30335

Vérification du nombre de lignes dans les tables après le chargement pour garantir que toutes les données des fichiers CSV ont bien été importées



The screenshot shows a database interface with a sidebar on the left listing the columns of the 'Region' table. The main area displays a SQL query and its result.

Table
Contrat
Region

Column
Code_dep_code_co...
reg_code INTEGER
reg_nom TEXT
aca_nom TEXT
dep_nom TEXT
com_nom_maj_court...
dep_code TEXT
dep_nom_num TEXT

```
1 SELECT COUNT(*) FROM Region;
```

! COUNT(*)
38916

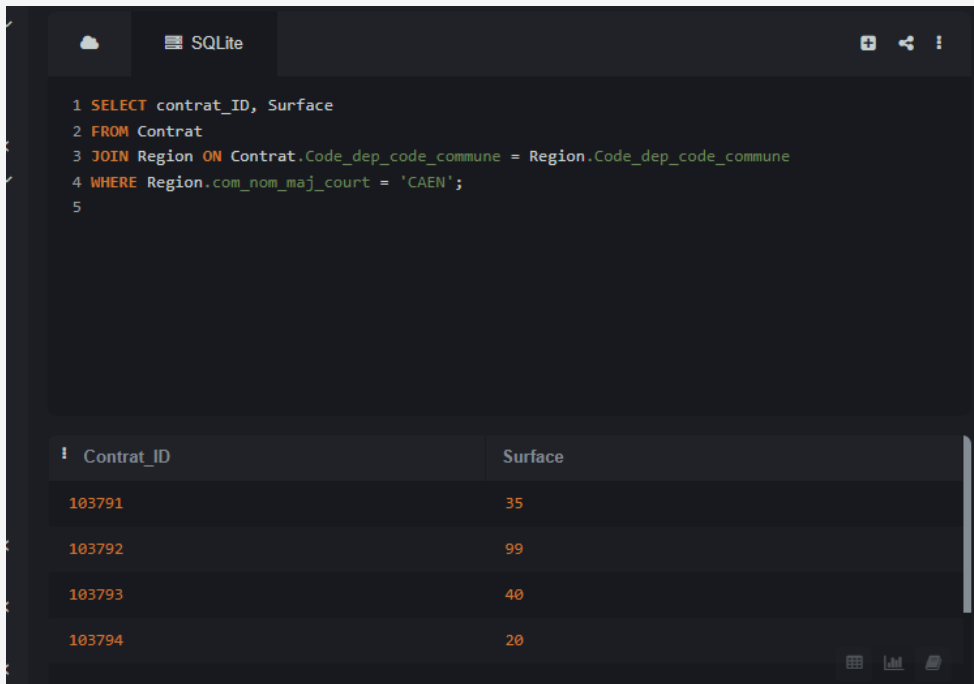
Requêtes: **SELECT**
COUNT(*) FROM Region;

SELECT COUNT(*) FROM
Contrat;

Rédactions des requêtes SQL

Rédaction des 12 requêtes SQL en m'appuyant sur les cours disponibles sur *Openclassrooms*, *Codecademy* et *SQL.sh* ,
en m'assurant que un résultat existe pour chaque requête et qu'elles sont fonctionnelles (pas de message d'erreur).

Rédaction des trois premières requêtes pour se familiariser avec l'outil et les différentes commandes :



The screenshot shows a SQLite IDE interface. At the top, there's a tab labeled 'SQLite'. Below it, a text area contains the following SQL query:

```
1 SELECT contrat_ID, Surface
2 FROM Contrat
3 JOIN Region ON Contrat.Code_dep_code_commune = Region.Code_dep_code_commune
4 WHERE Region.com_nom_maj_count = 'CAEN';
5
```

Below the query editor, a table of results is displayed with two columns: 'Contrat_ID' and 'Surface'.

Contrat_ID	Surface
103791	35
103792	99
103793	40
103794	20

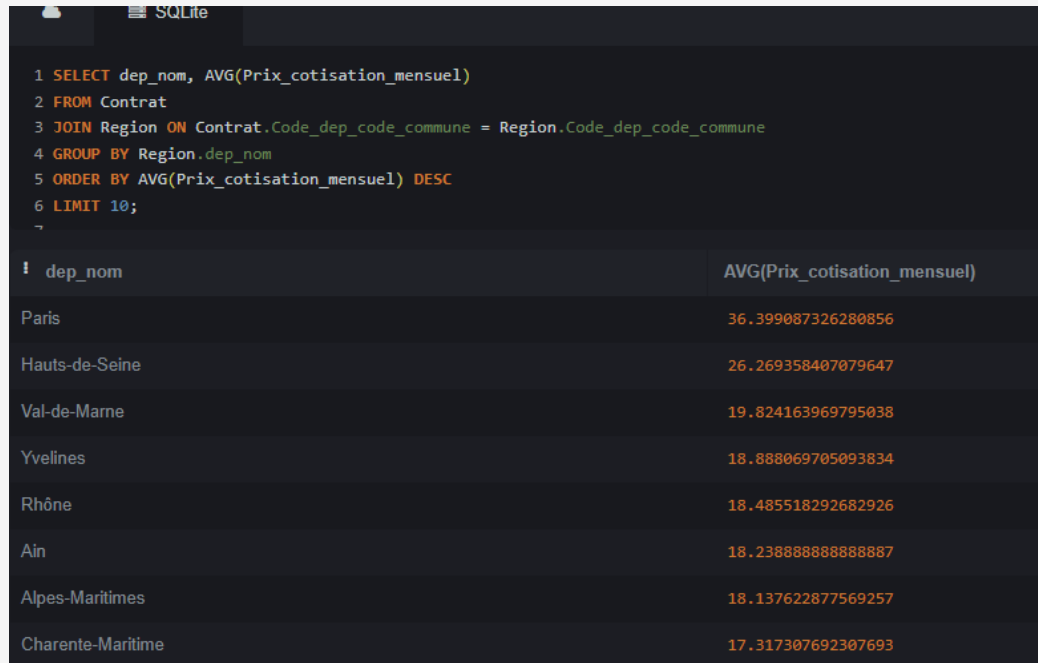
Ex: Requête 1 - Lister les numéros de contrats (contrat_ID) avec leur surface pour la commune de Caen.

Utilisation des commandes SELECT, FROM, JOIN, INNER JOIN, WHERE, ORDER BY,

Rédactions des requêtes SQL (suite)

Après avoir correctement effectué les premières requêtes, travail sur des analyses plus complexes en utilisant les commandes DESC, LIMIT, AVG, GROUP BY, ORDER BY, HAVING COUNT...

Utilisation de jointures entre les tables Contrat et Région pour croiser les données clients et géographiques. Exemples Requête n° 10 et 11 ci-dessous



SQLite

```
1 SELECT dep_nom, AVG(Prix_cotisation_mensuel)
2 FROM Contrat
3 JOIN Region ON Contrat.Code_dep_code_commune = Region.Code_dep_code_commune
4 GROUP BY Region.dep_nom
5 ORDER BY AVG(Prix_cotisation_mensuel) DESC
6 LIMIT 10;
```

dep_nom	AVG(Prix_cotisation_mensuel)
Paris	36.399087326280856
Hauts-de-Seine	26.269358407079647
Val-de-Marne	19.824163969795038
Yvelines	18.888069705093834
Rhône	18.485518292682926
Ain	18.238888888888887
Alpes-Maritimes	18.137622877569257
Charente-Maritime	17.317307692307693

```
1 SELECT com_nom_maj_court, COUNT(Contrat_ID)
2 FROM Contrat
3 JOIN Region ON Contrat.Code_dep_code_commune = Region.Code_dep_code_commune
4 GROUP BY com_nom_maj_court
5 HAVING COUNT(Contrat_ID) >= 150;
```

com_nom_maj_court	COUNT(Contrat_ID)
BORDEAUX	302
COURBEVOIE	163
GRENOBLE	220
LILLE	161
NANTES	291
NICE	387
PARIS 3	159
PARIS 9	204
PARIS 10	263

CONCLUSION

A travers ce projet J'ai appris qu'il est essentiel de bien comprendre la structure et le contenu des fichiers de données avant tout analyse.

J'ai découvert les différentes étapes d'une modélisation de base de données, ainsi que la notion de redondance, qu'il faut éviter à tout prix. Acquis la capacité de créer mes premiers diagrammes de classe UML.

A construire un schéma relationnel et utiliser la modélisation relationnelle, construire des requêtes SQL simples et appliquer d'autres fonctionnalités aux requêtes SQL. Ces acquis me serviront pour les projets avenir.